



SIMULACIONES Y PROCESOS ESTOCÁSTICOS

2026

Teoría: Dr. Alejandro Mezio – mezioalejandro@uca.edu.ar

Práctica: Dr. Luis Herrera – lherrera@uca.edu.ar

A decorative graphic on the left side of the slide, consisting of a network of light blue lines and small circles, resembling a circuit board or a neural network, extending from the top to the bottom.

MÉTODOS DE OPTIMIZACIÓN

- G. Ciaburro, Hand-os Simulations...: Capítulo 7

Problemas de Optimización:

$$\min f(x) \quad \forall x \in F$$

- f : función objetivo
- F : espacio donde vive x
- Buscamos el mínimo global. No existe x donde el valor de $f(x)$ sea menor.
- Si el problema es de maximización, cambiamos el signo de f

Dificultades:

- La función objetivo usualmente es no lineal y compleja
- Pueden existir mínimos locales que dificulten avanzar hacia un mínimo global
- El espacio F puede tener una topología que vuelva complicado encontrar el punto deseado (por ejemplo, no ser compacto)

Problemas de Optimización:

En Machine Learning la función objetivo es de manera general una función pérdida (o costo):

Supongamos un modelo que recibe como datos de entrada d variables (la cantidad de features son d), cada fila de entrada x_i tiene d valores (los datos de entrada tienen d columnas). También llamemos y_i las variables target (puede ser un número, una etiqueta, un booleano).

Si llamamos $\mathbf{w} = \{w_1, w_2, \dots, w_d\}$ a los parámetros con los que queremos entrenar nuestro modelo, $\mathbf{w} \in \mathbb{R}^d$, y la función pérdida es una cantidad de la forma,

$$L(\mathbf{w}) = \frac{1}{n} \sum_{i=1}^n \ell(f_{\mathbf{w}}(x_i), y_i)$$

Por ejemplo:

$$(f_{\mathbf{w}}(x_i) = \hat{y}_i)$$

- Regresión $\rightarrow \ell(\hat{y}_i, y_i) = (\hat{y}_i - y_i)^2$ **(MSE)**

- Clasificación $\rightarrow \ell(\hat{y}_i, y_i) = -y_i \log(\hat{y}_i) - (1 - y_i) \log(1 - \hat{y}_i)$ **(Cross-Entropy)**

$$\ell(\hat{y}_i, y_i) = \begin{cases} -\log(\hat{y}_i), & \text{si } y_i = 1 \\ -\log(1 - \hat{y}_i), & \text{si } y_i = 0 \end{cases}$$

Problemas de Optimización:

$$L(w) = \frac{1}{n} \sum_{i=1}^n \ell(f_w(x_i), y_i)$$

- La idea es que mientras más correcta sea la predicción, más pequeña o cercana a cero será la función pérdida
- Notemos que la suma en i es una suma sobre todos los datos de entrada (sobre todas las filas del dataframe)

Para controlar el overfitting suele agregarse un término de “regularización”

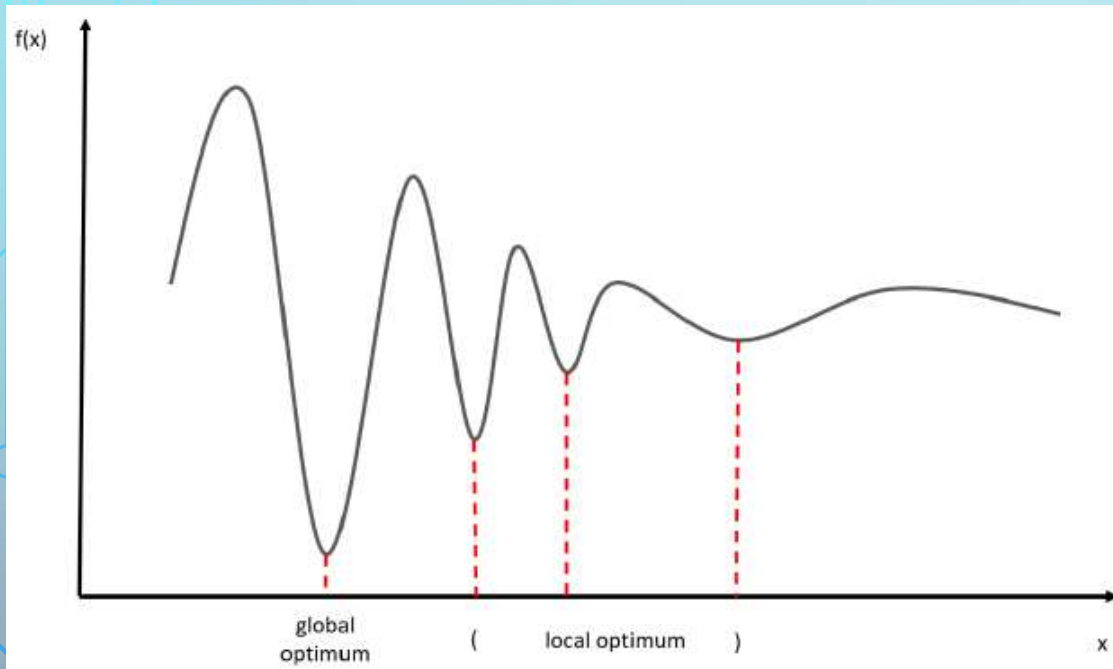
$$L(w) = \frac{1}{n} \sum_{i=1}^n \ell(f_w(x_i), y_i) + \lambda R(w)$$

Entonces, lo que buscamos son los valores de w que minimicen $L(w)$, es decir:

$$w^* = \arg \min_w L(w)$$

Problemas de Optimización:

Es fácil ver las posibles dificultades en 1 dimensión (un solo w):



- Mínimo global



- Mínimo local



En ambos casos, la derivada de $L(w)$ (o de $f(x)$) se anula

Problemas de Optimización:

Recordemos el problema de punto fijo (en 1D):

- Resolvemos $x = f(x)$ de manera iterativa
- Tomamos un valor inicial x_0
- $x_{n+1} = f(x_n)$
- Y llegamos a destino cuando esta iteración no cambia mucho

También hicimos la siguiente observación:

- $x_{n+1} = x_n + (f(x_n) - x_n)$
- $x_{n+1} = x_n + \Delta x_n$
- *Valor nuevo = valor viejo + lo que corrige al valor viejo*

Y así nos armamos una versión mas “suavizada”:

- $x_{n+1} = x_n + \alpha \Delta x_n$

Con tasa de aprendizaje (learning rate) α

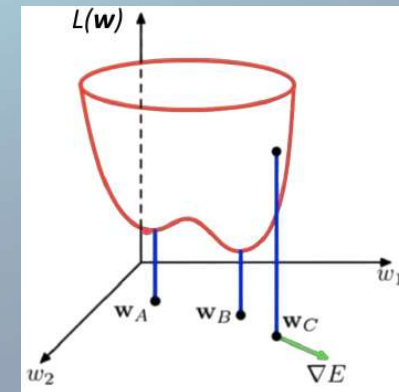
Ahora, buscamos $\mathbf{w}$ que minimice L :

$$\mathbf{w}^{(n+1)} = \mathbf{w}^{(n)} + \Delta \mathbf{w}^{(n)}$$

(notar cambio de notación. $\mathbf{w}$ es un vector y no queremos confundir iteración con componentes)

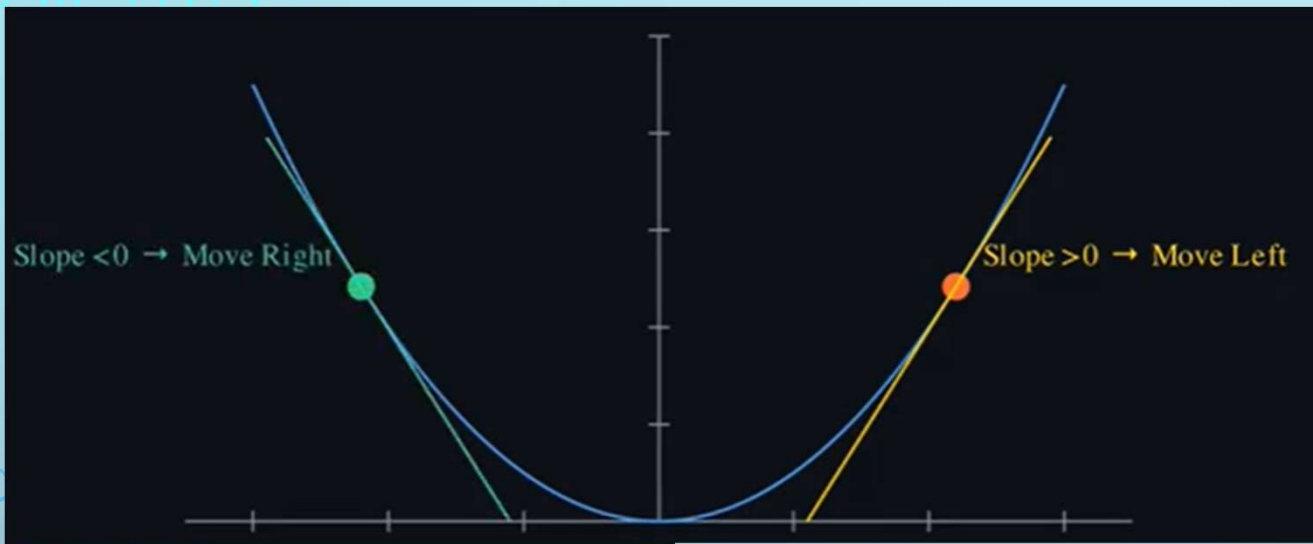
Variantes:

- Gradiente descendiente
- Gradiente descendiente estocástico
- Newton-Raphson

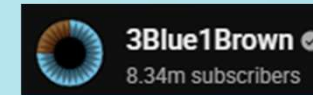


Gradiente Descendiente

Nuevamente, se entiende fácil en 1D:



- La derivada nos indica hacia dónde hay que moverse para acercarnos al mínimo
- Su valor también nos indica cuánto decrece la función cuando nos movemos hacia el mínimo
- $w^{(n+1)} = w^{(n)} - \alpha \frac{dL}{dw}(w^{(n)})$
- El algoritmo convergerá al w^* deseado cuando $\frac{dL}{dw}(w^*) = 0$

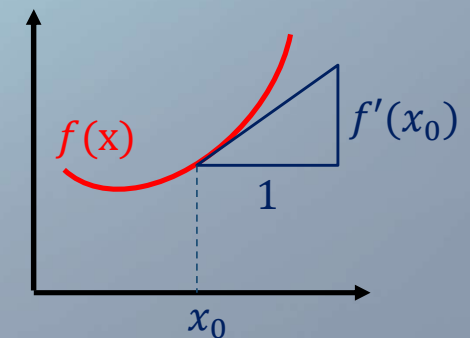


Gradient descent, how neural networks learn

<https://www.youtube.com/watch?v=IHZwWFHWa-w>

Más recursos interesantes para entender:

- <https://studiotvai.com/gradient-descent>
- <https://medium.com/data-science/a-visual-explanation-of-gradient-descent-methods-momentum-adagrad-rmsprop-adam-f898b102325c>
- <https://www.youtube.com/watch?v=jgRAhqlqK8s>



Gradiente Descendiente

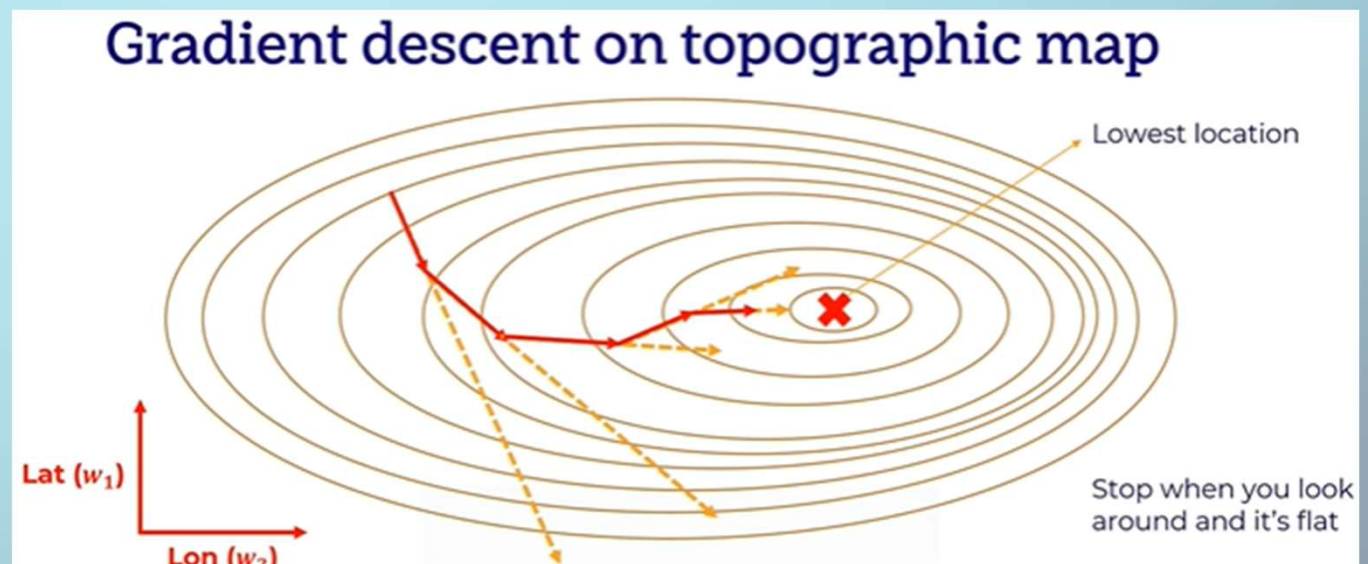
Para muchas (d) dimensiones: $w^{(n+1)} = w^{(n)} - \alpha \nabla L(w^{(n)})$

Si $L(w)$ es diferenciable, habrá mínimo en $\nabla L(w^*) = 0$

Otra forma de resolver esto es encontrando el cero de ∇L (Newton-Raphson)

Importancia del α :

- **Muy pequeño:** convergencia lenta, tardamos mucho en llegar al mínimo
- **Muy grande:** mucha oscilación, puede no converger nunca
- **Valor apropiado:** convergencia estable y buen balance costo/precisión



Ver notebook 08_optimizacion_Claude.ipynb
y Ciaburro

Gradiente Descendiente

Función pérdida:
$$L(w) = \frac{1}{n} \sum_{i=1}^n \ell(f_w(x_i), y_i) + \lambda R(w)$$

- Sumo sobre todos los datos (¡muchos!)
- Las componentes de x_i pueden tener diferentes naturaleza $\rightarrow L(w)$ muy complicada
- $\ell(\hat{y}_i, y_i)$ puede ser complicada y no tenemos una expresión fácil para evaluar ∇L

Hay muchas variantes:

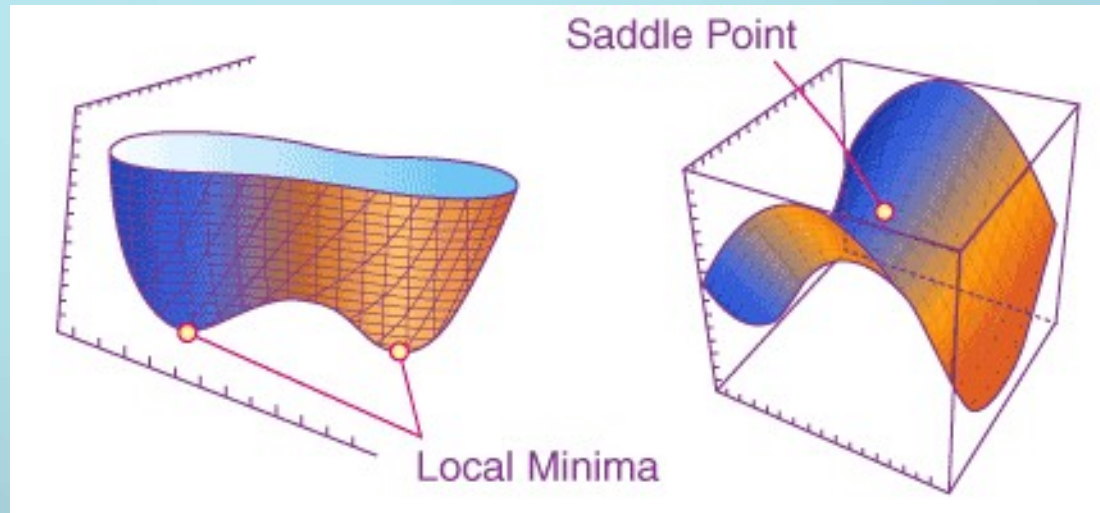
- **Batch GD:** es lo que describimos hasta ahora. Usamos todos los datos, usamos todo el batch.
Es estable, pero resulta caro si tenemos muchos datos
- **Stochastic GD:** Usa un solo dato x_i seleccionado aleatoriamente en cada iteración.
Resulta muy ruidoso, pero nos permite escapar de posibles mínimos locales.
Es la base del deep-learning
- **Mini-batch SGD:** Similar a SGD pero usa pequeños lotes de datos (en vez de un solo dato).
Termina siendo una opción balanceada entre estabilidad (Batch GD), velocidad y evitar mínimos locales (SGD)
En la práctica está muy estandarizado su uso

Ver notebook [08_optimizacion_Claude.ipynb](#)
y Ciaburro

Gradiente Descendiente

Problemas: w^* donde $\nabla L(w^*) = 0$ pero no es mínimo global

- **Mínimos locales**
- **Puntos de ensilladura**
- **Valles planos**



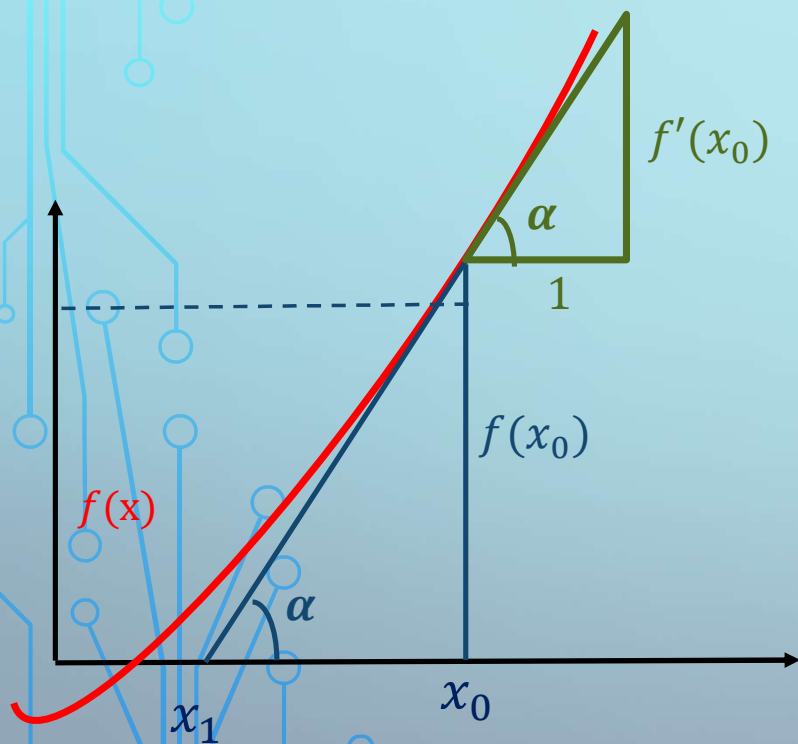
OTRO MÉTODO $\rightarrow$ buscamos w^* directamente buscando dónde ocurre que $\nabla L(w^*) = 0$

NEWTON-RAPHSON

Newton-Raphson

Explicuemos nuevamente con 1D

El método de Newton-Raphson es un método para encontrar la x tal que $f(x) = 0$:



- Pensemos en la derivada de f en el punto x_0
- Miremos el **triángulo rectángulo** de base 1 y altura $f'(x_0)$
- Nos permite pensar en qué es una derivada

$$\tan \alpha = \frac{f'(x_0)}{1}$$

- Y ahora miremos el **triángulo equivalente** de base $(x_0 - x_1)$ y altura $f(x_0)$. El ángulo (y su tangente) es idéntico:

$$\tan \alpha = \frac{f(x_0)}{(x_0 - x_1)} = f'(x_0)$$

Reacomodando:
$$x_1 = x_0 - \frac{f(x_0)}{f'(x_0)}$$

Newton-Raphson

Fórmula iterativa: $x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$

Recordemos que lo que buscábamos originalmente era hacer cero la derivada de una función

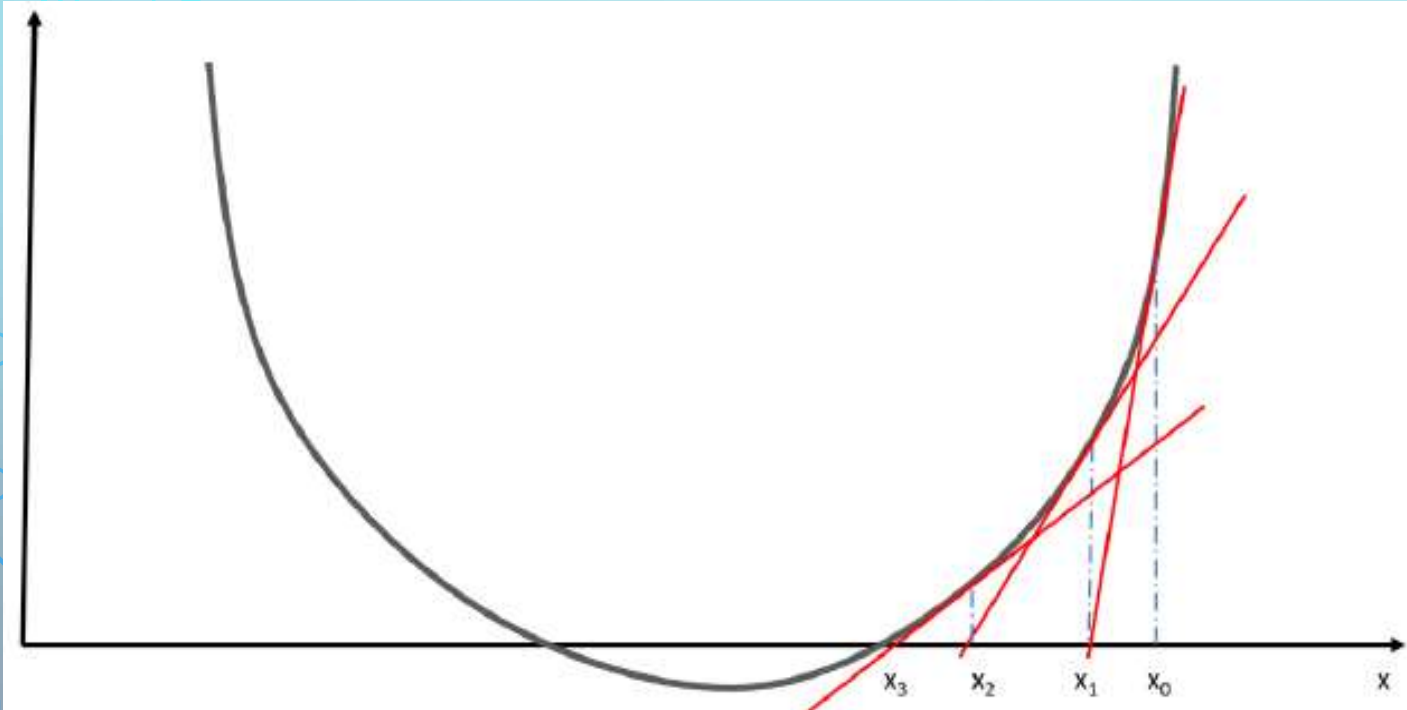
$$f \rightarrow f'$$

$$f' \rightarrow f''$$

Y la fórmula iterativa

$$x_{n+1} = x_n - \frac{f'(x_n)}{f''(x_n)}$$

nos permite acercarnos a la solución de $f'(x) = 0$



Newton-Raphson

Para dimensiones mayores, queremos encontrar w^* tal que $\nabla L(w^*) = 0$

El gradiente ∇L es un vector, su derivada se denomina Hessiano $H_L(w)$ y es una matriz

$$w^{n+1} = w^n - [H_L(w^n)]^{-1} \cdot \nabla L(w^n)$$

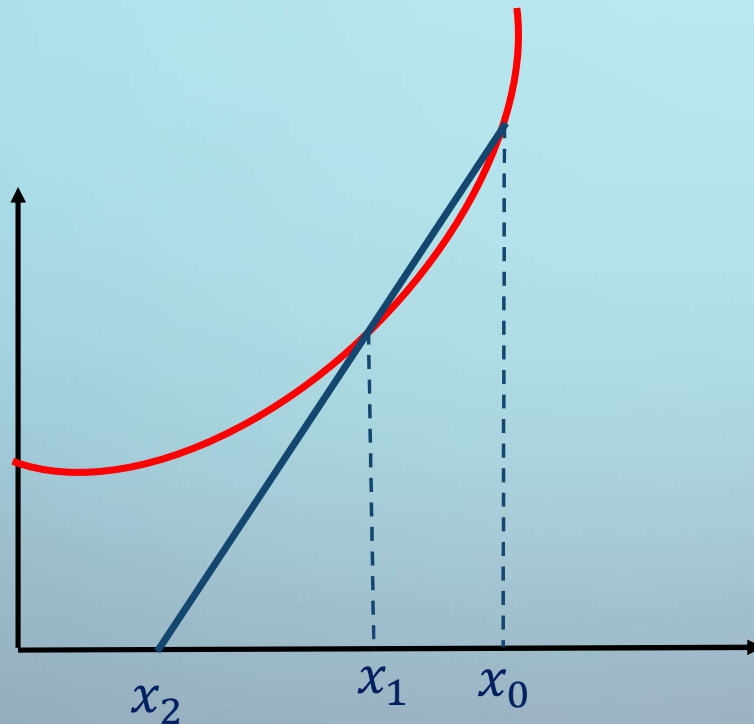
La interpretación geométrica de Newton-Raphson aplicado a ∇L es que estamos aproximando a la función L con una función cuadrática de w

Contra: para dimensiones grandes es muy costoso calcular el Hessiano

Ver notebook [08_optimizacion_Claude.ipynb](#)
Y Ciaburro

Newton-Raphson

Comentario al pasar: Existe una variante llamada “método de la secante”



No me armo x_{n+1} con la derivada en x_n

Me armo x_{n+1} con una recta secante que une $f(x_n)$ y $f(x_{n-1})$

Estimación por Máxima Verosimilitud

(Maximum Likelihood Estimation, MLE)

StatQuest videos

<https://www.youtube.com/watch?v=pYxNSUDSFH4>

<https://www.youtube.com/watch?v=XepXtl9YKwc>

Pensemos en una clasificación binaria:

Para cada fila de datos $x_i = (x_{i,1}, x_{i,2}, \dots, x_{i,d})$ tenemos un target y_i que puede ser 1 o 0 (éxito o fracaso).

Luego tenemos un modelo estadístico (una distribución estadística) caracterizada por d valores $w = \{w_1, w_2, \dots, w_d\}$ que para cada fila de datos nos da una probabilidad que nos dice cuán probable es que la fila i pertenezca al 1 (éxito),

$$x_i \rightarrow \hat{y}_i = p(x_i|w)$$

La predicción $\hat{y}_i$ se lee como la probabilidad de que x_i sea éxito, dado el modelo estadístico que tiene parámetros $w = \{w_1, w_2, \dots, w_d\}$

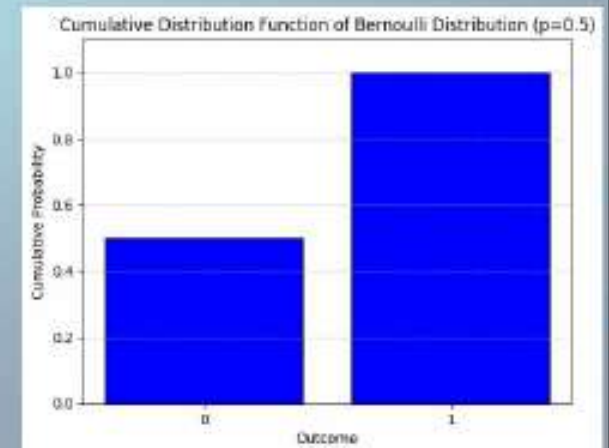
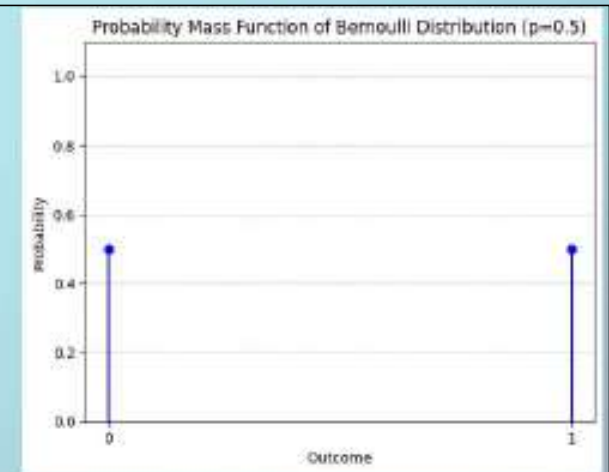
- Cuando $y_i = 1$ quisiéramos que nuestro modelo obtenga un $\hat{y}_i$ cercano a 1. $p(x_i|w)$ lo mas alta posible
- Cuando $y_i = 0$ quisiéramos que nuestro modelo obtenga un $\hat{y}_i$ cercano a 0. $p(x_i|w)$ lo mas baja posible

Estimación por Máxima Verosimilitud

Recordando de la clase de probabilidad y estadística...

BERNOULLI *(¡lanzar moneda!)*

- Experimento cuyo resultado sólo toma 2 valores
- Éxito es $X = 1$ y ocurre con probabilidad p
- Fracaso es $X = 0$ y probabilidad $1 - p$
- $P(X = 1) = p$ y $P(X = 0) = 1 - p$
- $P(X = x) = p^x(1 - p)^{1-x}$, con $x \in \{0,1\}$
- Valor esperado $E[X] = p$
- Varianza $Var(X) = p(1 - p)$



Estimación por Máxima Verosimilitud

Veamos con más detalle la distribución de Bernoulli:

$$P(X = x) = p^x(1 - p)^{1-x}$$

donde x es 0 o 1 (fracaso o éxito), y p es la probabilidad de obtener 1 (éxito)

Recordemos que esta cantidad P nos dice cuán probable es obtener el resultado x al tirar una moneda, dado que esa moneda tenga una distribución de probabilidad p para éxito y $(1 - p)$ para fracaso.

En la situación actual, cada fila de datos x_i y su predicción $\hat{y}_i$ son un evento (tiro una moneda). El resultado que estamos buscando será $y_i = 1$ (éxito) o $y_i = 0$ (fracaso), nuestro modelo nos da un número $\hat{y}_i$ entre 0 y 1 que es nuestra aproximación actual. Entonces podemos pensar que la probabilidad de Bernoulli $P(X = x)$ mide la probabilidad de que nuestro modelo $\hat{y}_i$ aproxime bien al target y_i .

Estimación por Máxima Verosimilitud

Así, para la fila de datos x_i y con nuestro modelo con parámetros w , la cantidad

$$\begin{aligned} P(X = y_i) &= p(x_i|w)^{y_i} (1 - p(x_i|w))^{1-y_i} \\ &= \hat{y}_i^{y_i} (1 - \hat{y}_i)^{1-y_i} \end{aligned}$$

mide que tan bien nuestro modelo probabilístico se acerca al resultado correcto.

En otras palabras, dado un modelo probabilístico con parámetros w , al cual le damos los datos de entrada x_i y nos devuelve $\hat{y}_i$, $P(X = y_i)$ nos dice la probabilidad de que nuestra predicción sea **similar** al target.

- Si $y_i = 1$ y obtengo $\hat{y}_i \approx 1 \rightarrow P(X = y_i) \approx 1$
- Si $y_i = 1$ y obtengo $\hat{y}_i \approx 0 \rightarrow P(X = y_i) \approx 0$
- Si $y_i = 0$ y obtengo $\hat{y}_i \approx 1 \rightarrow P(X = y_i) \approx 0$
- Si $y_i = 0$ y obtengo $\hat{y}_i \approx 0 \rightarrow P(X = y_i) \approx 1$

Este $P(X = y_i)$ es la llamada verosimilitud de nuestro modelo para la fila de datos x_i

Estimación por Máxima Verosimilitud

Si nuestro entrenamiento es con N filas de datos, obtenemos la verosimilitud total multiplicando la verosimilitud de cada fila (ya que mide una probabilidad):

$$L(w) = \prod_{i=1}^N P(X = y_i) = \prod_{i=1}^N \hat{y}_i^{y_i} (1 - \hat{y}_i)^{1-y_i}$$

“ L ” de likelihood (aunque use la misma notación que antes usamos para la pérdida “loss”)

- Podemos pensar $L(w) = P(\text{datos} \mid w)$ como la probabilidad (condicional) de que reproduzcamos correctamente los “datos” teniendo un modelo con parámetros w
- Esta función de verosimilitud es la que tenemos que maximizar (en vez de minimizar) para conseguir el modelo que mejor reproduzca los datos

Problema: estamos multiplicando muchos números pequeños (muy inestable computar)

Estimación por Máxima Verosimilitud

$$\begin{aligned}\ln(a \cdot b) &= \ln a + \ln b \\ \ln(a^b) &= b \cdot \ln a\end{aligned}$$

Solución → aplicar logaritmo

- Transforma multiplicaciones en sumas
- Transforme potencias en multiplicaciones
- Magnitudes que abracan diferentes escalas pueden ser comparadas

$$\begin{aligned}\mathcal{L}(w) = \ln L(w) &= \sum_{i=1}^N [\ln \hat{y}_i^{y_i} + \ln(1 - \hat{y}_i)^{1-y_i}] \\ &= \sum_{i=1}^N [y_i \cdot \ln \hat{y}_i + (1 - y_i) \cdot \ln(1 - \hat{y}_i)]\end{aligned}$$

Entonces, lo que buscamos son los valores de w que maximicen la log-likelihood $\mathcal{L}(w)$:

$$w^* = \arg \max_w \mathcal{L}(w)$$

Estimación por Máxima Verosimilitud

$$\mathcal{L}(w) = \sum_{i=1}^N [y_i \cdot \ln \hat{y}_i + (1 - y_i) \cdot \ln(1 - \hat{y}_i)]$$

Como los términos en corchetes son todos del mismo orden de magnitud, es común dividir por N para que el valor numérico no sea muy grande:

$$\mathcal{L}(w) = \frac{1}{N} \sum_{i=1}^N [y_i \cdot \ln \hat{y}_i + (1 - y_i) \cdot \ln(1 - \hat{y}_i)]$$

Notemos que podemos interpretarlo como el promedio de $[y_i \cdot \ln \hat{y}_i + (1 - y_i) \cdot \ln(1 - \hat{y}_i)]$

Conexión con redes neuronales

Como hemos dicho antes, para pasar de un problema de maximización a uno de minimización sólo hace falta cambiar el signo.

La función pérdida utilizada en redes neuronales es la que se conoce como “**negative log-likelihood**”, o bien “**cross-entropy**” (entropía cruzada):

$$S(w) = -\frac{1}{N} \sum_{i=1}^N [y_i \cdot \ln \hat{y}_i + (1 - y_i) \cdot \ln(1 - \hat{y}_i)]$$

- Originalmente propuesta por Shannon, la palabra entropía (un concepto de la física y la termodinámica) aparece acá simplemente porque tiene la misma forma funcional.

Entrenar una red neuronal con cross-entropy es igual a hacer MLE bajo el modelo probabilístico correspondiente

<https://towardsdatascience.com/cross-entropy-negative-log-likelihood-and-all-that-jazz-47a95bd2e81/>

Multiclase: ejemplo MNIST

- En vez de 2 clases (0 y 1), tenemos más categorías (10 en el caso de MNIST)
- Para cada fila de datos x_i los modelos de clasificación obtienen $\hat{y}_{i,k}$ para cada clase que representa la probabilidad de que esa fila pertenezca a la clase k

	X						y	Model's output									
	11x8	11x9	...	15x18	15x19	15x20	label	$\hat{y}_{i,0}$	$\hat{y}_{i,1}$	$\hat{y}_{i,2}$	$\hat{y}_{i,3}$	$\hat{y}_{i,4}$	$\hat{y}_{i,5}$	$\hat{y}_{i,6}$	$\hat{y}_{i,7}$	$\hat{y}_{i,8}$	$\hat{y}_{i,9}$
7	59	160	...	242	37	0	7	0.04	0.05	0.04	0.01	0.01	0.02	0.01	0.8	0.01	0.01
5	0	226	...	0	0	0	5	0.01	0.07	0.05	0.01	0.05	0.7	0.04	0.05	0.01	0.01
3	0	0	...	209	252	252	3	0.1	0.04	0.1	0.2	0.04	0.02	0.01	0.17	0.3	0.02
5	0	0	...	153	252	205	5	0.03	0.05	0.01	0.05	0.01	0.8	0.01	0.01	0.02	0.01

Cuando decimos que el target es $y_i = 7$, estamos hablando de un vector de 10 elementos $y_i = (y_{i,0}, y_{i,1}, \dots, y_{i,9})$ donde $y_{i,7} = 1$ y el resto de elementos es cero.

Multiclase: MNIST

Entonces tenemos,

label	y_0	y_1	y_2	y_3	y_4	y_5	y_6	y_7	y_8	y_9	$\hat{y}_{i,0}$	$\hat{y}_{i,1}$	$\hat{y}_{i,2}$	$\hat{y}_{i,3}$	$\hat{y}_{i,4}$	$\hat{y}_{i,5}$	$\hat{y}_{i,6}$	$\hat{y}_{i,7}$	$\hat{y}_{i,8}$	$\hat{y}_{i,9}$
7	0	0	0	0	0	0	0	1	0	0	0.04	0.05	0.04	0.01	0.01	0.02	0.01	0.8	0.01	0.01
5	0	0	0	0	0	1	0	0	0	0	0.01	0.07	0.05	0.01	0.05	0.7	0.04	0.05	0.01	0.01
3	0	0	0	1	0	0	0	0	0	0	0.1	0.04	0.1	0.2	0.04	0.02	0.01	0.17	0.3	0.02
5	0	0	0	0	0	1	0	0	0	0	0.03	0.05	0.01	0.05	0.01	0.8	0.01	0.01	0.02	0.01

Antes:

$$P(X = y_i) = \hat{y}_i^{y_i} (1 - \hat{y}_i)^{1-y_i}$$

Ahora:

$$P(X = y_i) = \prod_{k=0}^9 \hat{y}_{i,k}^{y_{i,k}}$$

y la cross-entropy queda:

$$S(w) = -\frac{1}{N} \sum_{i=1}^N \sum_{k=0}^9 [y_{i,k} \cdot \ln \hat{y}_{i,k}]$$

A decorative graphic on the left side of the slide, consisting of a network of light blue lines and small circles, resembling a circuit board or a neural network diagram.

Ver notebook de Ciaburro sobre la máxima verosimilitud

Ver notebook propuesto por Claude para ejemplificar



FIN